

情報学群実験第 4 音の信号と画像解析

学籍番号 1280391

細川 夏風

2026 年 8 月 11 日

1 はじめに

物理現象の解析には音と画像は必要不可欠な要素と言って過言ではない。理由は様々な状況において視覚と聴覚は得られた情報を後日になってから解析が可能になるからである。また、音と画像はフーリエ解析という方法により明確な解析が可能である。今実験では、フーリエ変換と通じて音や画像の持つ周波数的特性とそこから得られる情報を議論してく。

2 音の信号の解析

方法

本実験では、MATLAB を用いて音の工学的特徴について学習し、複数の信号生成および処理を実施した。各課題の具体的な手法を以下に示す。なお、本節の課題を実装した MATLAB プログラムについては、付録 A.1 に示す。

2.1 音の信号

2.1.1 課題 1；周波数の異なる純音の生成

サンプリング周波数 $F_s = 16000$ Hz、提示時間 1 秒間として、周波数が 440 Hz および 660 Hz の純音 (正弦波) を生成した。正弦波の生成には以下の式を用いた。

$$y(t) = \sin(2\pi ft)$$

生成した信号は `sound` 関数を用いて再生し、波形が確認できるようにグラフに描画した。

2.1.2 課題 2：振幅・位相の確認

課題 1 で作成した 440 Hz の純音を基準とし、振幅および初期位相を変更した信号を生成した。振幅は 0.5 倍および 0.25 倍の 2 種類、初期位相は 90 度 ($\frac{\pi}{2}$ ラジアン) および 180 度 (π ラジアン) の 2 種類作成した。信号の式は以下のように表される。

$$y(t) = \sin(2\pi ft + \phi)$$

各条件における波形の違いを比較するため、グラフをプロットした。

2.1.3 課題 3：うなり

サンプリング周波数 $F_s = 16000$ Hz、提示時間 4 秒間とし、周波数が 440 Hz と 441 Hz の 2 つの純音を加算合成した。周波数の差によって生じるうなりの現象を確認するため、合成波形を描画および再生した。

2.1.4 課題 4：フーリエ級数展開

代表的な波形が三角関数の加算合成により再現できることを確認するため、矩形波の生成を行った。加算する高調波の数 N を 1, 5, 25 と変化させ、以下の式を用いて合成した。

$$f(t) = \sum_{k=1}^N \frac{1}{2k-1} \sin(2\pi(2k-1)ft)$$

N を増やすことで波形が次第に矩形波へ収束していく過程をグラフによって確認した。

2.1.5 課題 5：白色ガウス雑音の生成

サンプリング周波数 $F_s = 16000$ Hz、提示時間 1 秒間として、平均 0、標準偏差 0.2 の白色ガウス雑音を生成した。MATLAB の `randn` 関数を用いて擬似乱数を生成し、その信号波形を描画した。さらに、生成した信号の振幅分布を確認するため、50 ビンに分割したヒストグラムを作成した。

2.1.6 発展課題

上記の基本課題に加えて、以下の発展的な検証を実施した。

- 課題 1 で生成した 440 Hz の純音データに対し、サンプリング周波数を 15000 Hz に変更して再生し、再生音の変化を確認した。
- サンプリング周波数 8000 Hz において、0.5 秒間で瞬時周波数が 440 Hz から 880 Hz へと線形に増大するチャープ波を生成し、その波形と音の変化を確認した。
- 課題 3 の差周波数を 2 Hz, -2 Hz, および 10 Hz にそれぞれ変更し、結果の波形や再生音の違いを比較した。
- 課題 4 のフーリエ級数展開の手法を応用し、三角波およびノコギリ波の加算合成を行った。

2.2 音のフーリエ解析

本節における課題 1 および課題 2 の周波数解析およびフィルタリング処理を実装した MATLAB プログラムについては、付録 A.2 に示す。

2.2.1 課題 1：正弦波と矩形波の周波数解析

サンプリング周波数 $F_s = 8192$ Hz において、正弦波および矩形波を生成し、高速フーリエ変換 (FFT) を用いて周波数解析を行った。正弦波は周波数 440 Hz、提示時間 1 秒間として生成した。矩形波は、128 点の 0 と 128 点の 1 を 4 回繰り返す合計 1024 点のデータとして生成した。それぞれの信号に対して `fft` 関数によりフーリエ変換を適用し、`fftshift` 関数を用いて負の周波数から正の周波数の順に並び替えた。その後、データの全時間長から算出した周波数軸を用いて、振幅スペクトルを描画した。

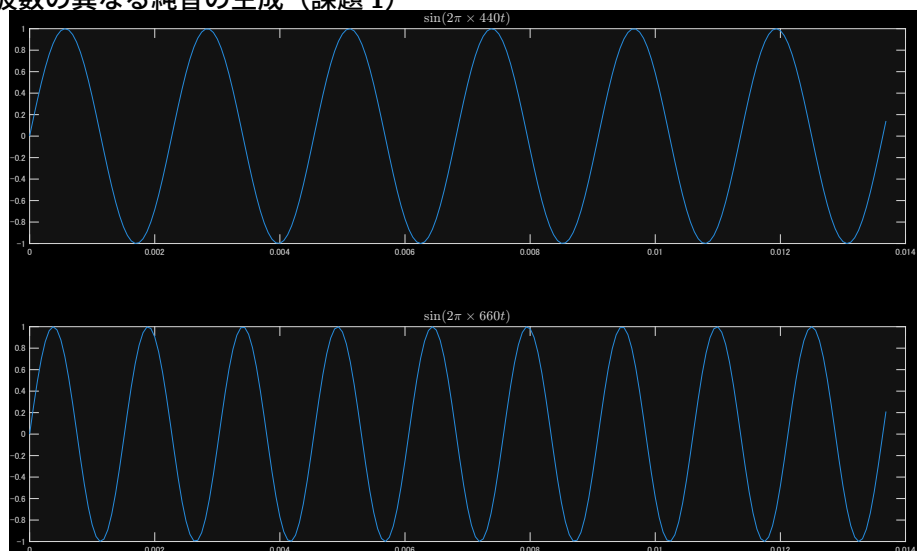
2.2.2 課題 2：メロディの生成と高域通過フィルタの適用

サンプリング周波数 $F_s = 8192$ Hz において、C(ド) から hi_C(高いド) までの 8 音を、それぞれ 0.25 秒間ずつ生成した。これら 8 つの音の配列を時間方向に連結して 1 つのメロディ波形を作成し、FFT および `fftshift` を用いて周波数領域のデータへ変換した。次に、高域通過フィルタ (HPF) を適用するため、カットオフ周波数を 357 Hz に設定した。周波数軸の配列を参照し、357 Hz 未満となる要素の周波数成分を 0 に書き換えることで低域成分を遮断した。最後に、フィルタ適用後のデータに対し `ifftshift` を行った後、`ifft` 関数を用いて逆フーリエ変換を適用し、時間領域の波形へ復元して結果を描画した。

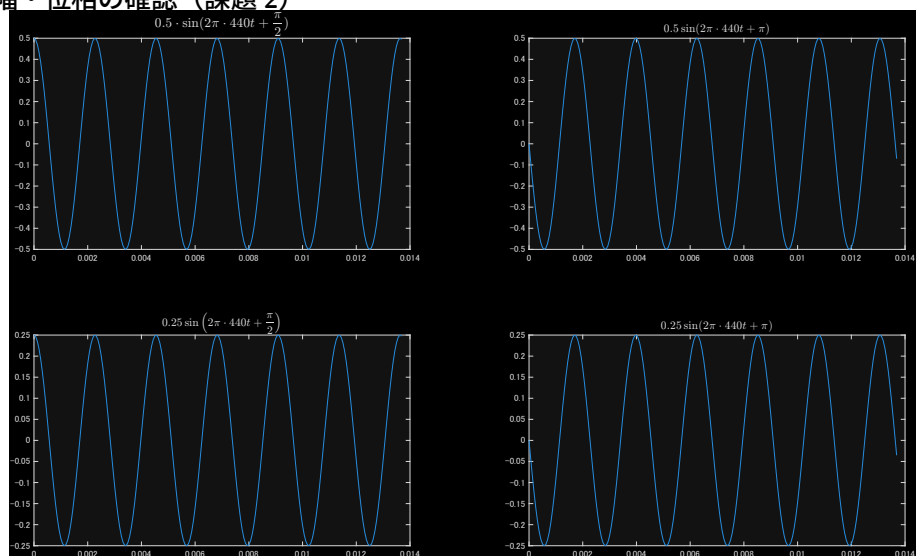
結果

2.3 音の信号の解析

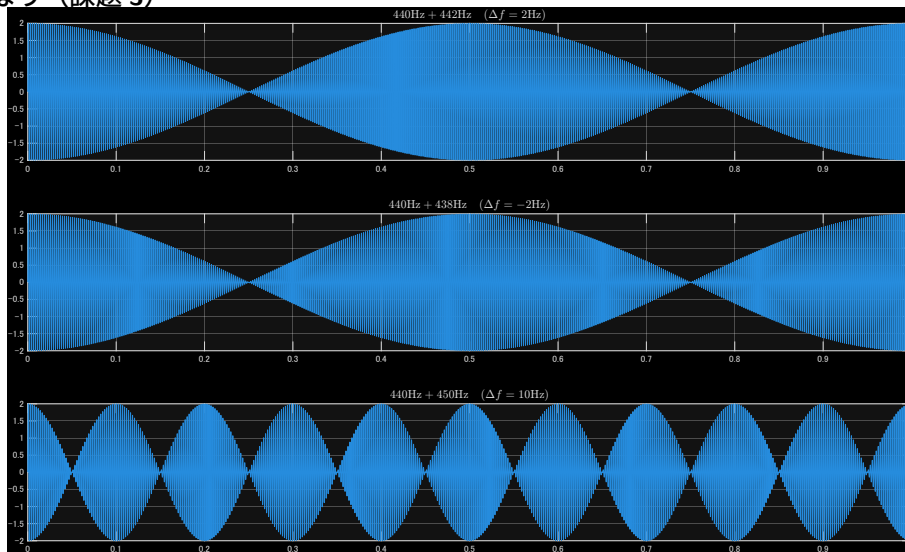
2.3.1 周波数の異なる純音の生成（課題 1）



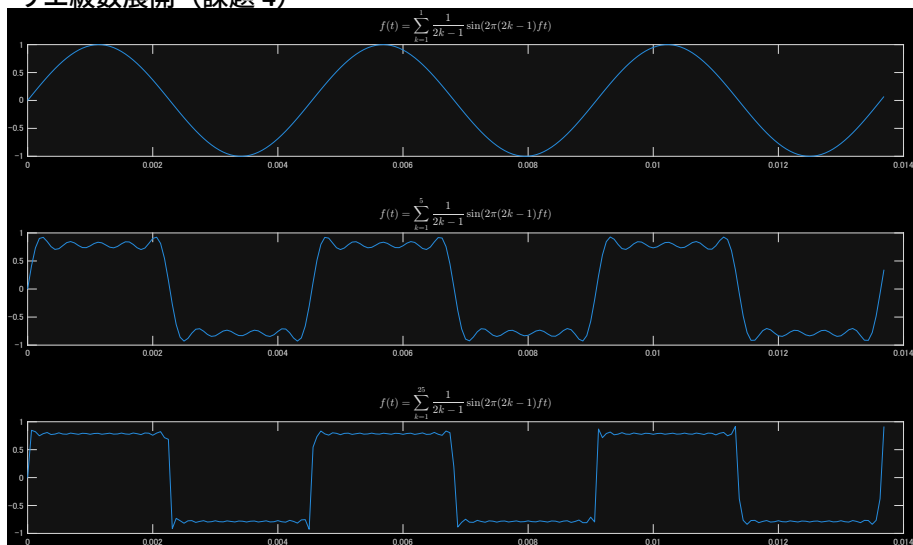
2.3.2 振幅・位相の確認（課題 2）



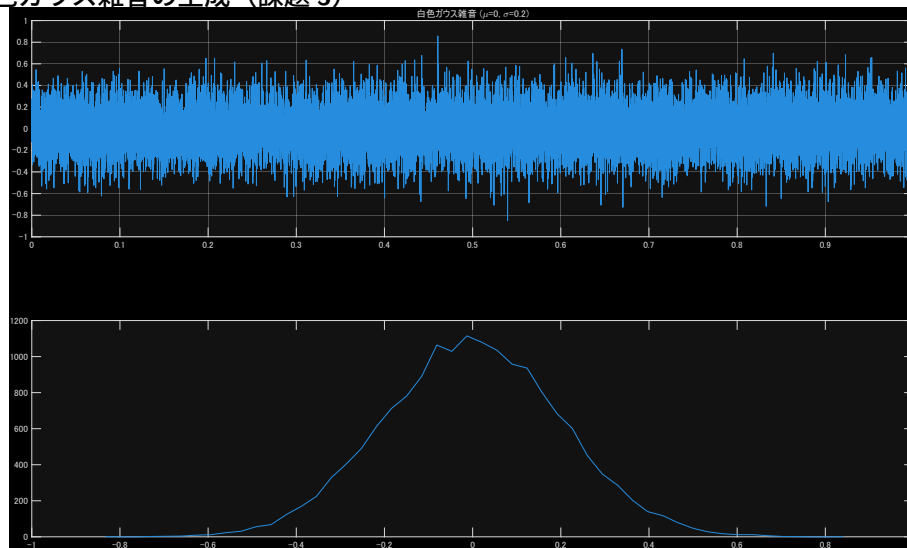
2.3.3 うなり（課題 3）



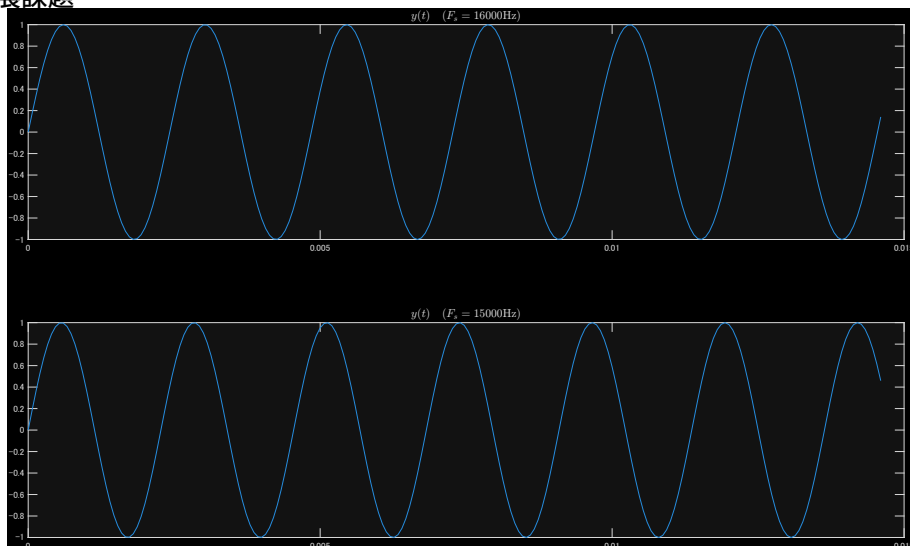
2.3.4 フーリエ級数展開（課題 4）

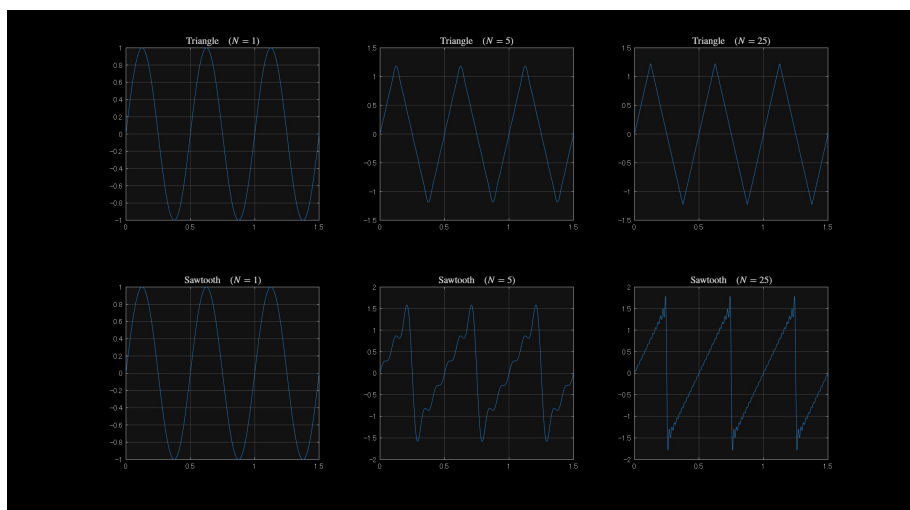
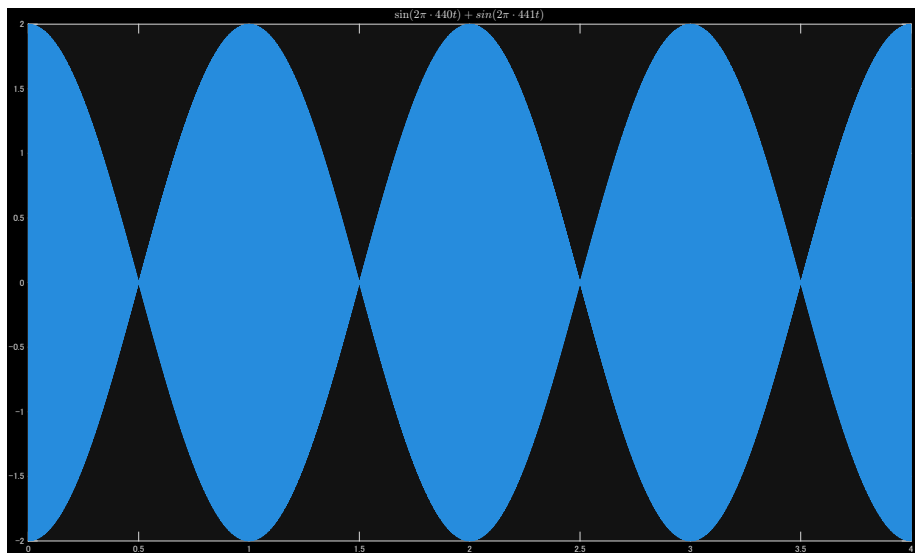
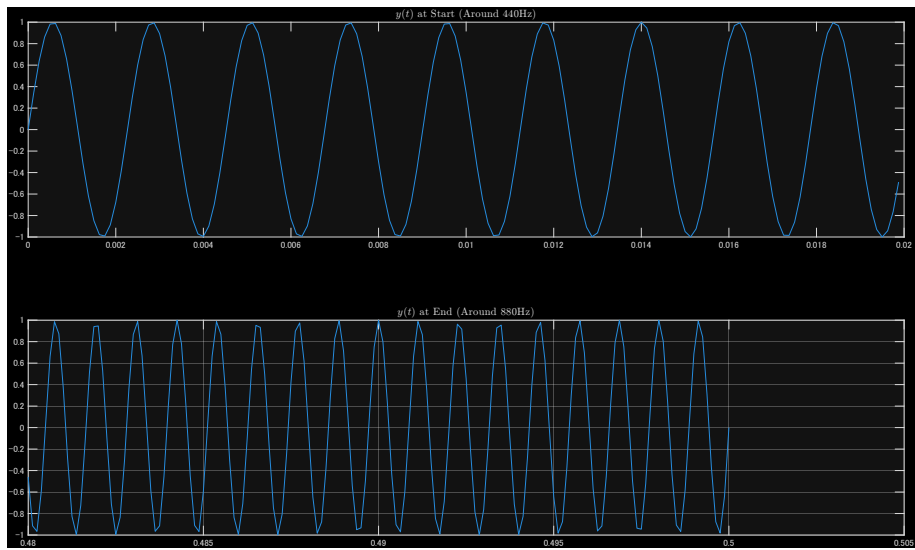


2.3.5 白色ガウス雑音の生成（課題 5）



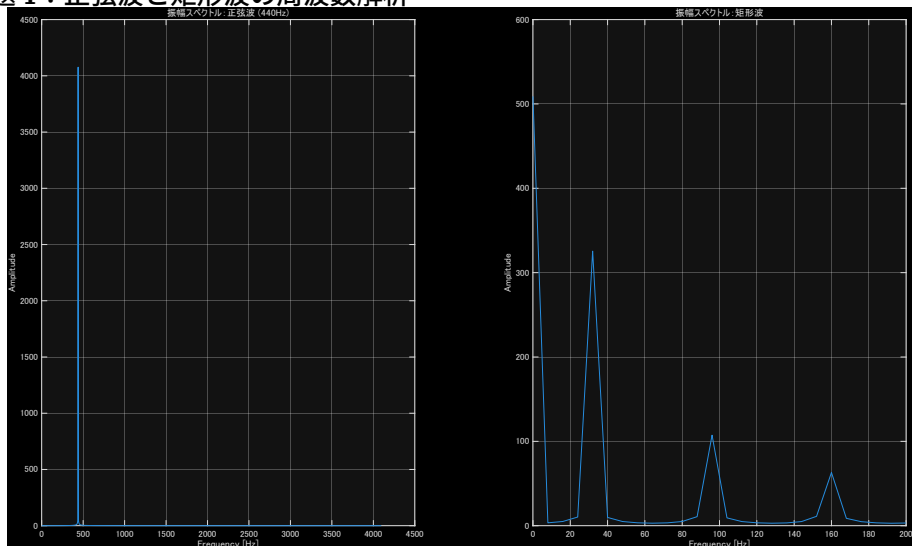
2.3.6 発展課題



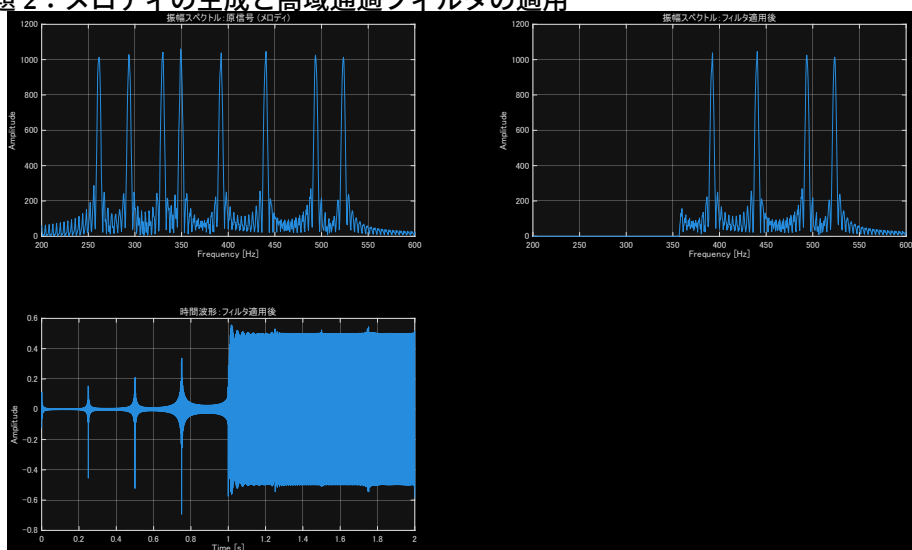


2.4 音のフーリエ解析

2.4.1 課題 1：正弦波と矩形波の周波数解析



2.4.2 課題 2：メロディの生成と高域通過フィルタの適用



考察

2.5 信号生成と解析

節 2.2.2 から、音の信号において周波数は周期の細かさといえる。その理由として図から 440Hz と 660Hz を比べた時、660Hz の方が周期が細かい事がわかる。また、それだけでなく信号に対しての積は振幅に影響することもわかる。これは信号、つまり周期関数を取る値に対しての積と同じ動作をしているからと言える。つまり下記の式と同様の操作を行っているといえる。

$$f(t) = \sin(2\pi ft)$$

$$g(t) = 0.5 \times \sin(2\pi ft)$$

$$g(t) = 0.5 \times f(t)$$

違う周期を持つ周期関数の和はその差についてズレの周期が異なるため、正負の取る値の順序が異なる。したがって打ち消し合う x 座標と助長し合う x 座標によって値の変動が起こっている事がわかる。この周期の差が 0 でなくかつ 0 に近い程、打ち消しや助長が少ない。周期の差が大きい場合はその逆であることがわかる。フーリエ級数展開とは周期関数、 $\sin, \cos$ の関数で概形を近似することであり、この項が無限の場合は関数が同値になる。この関数の項の増加に伴い関数の概形が正確になることがわかる。白色ガウス雑音について、平均を 0、標準偏差を 0.2 の乱数によりノイズを生成している。この“白色”についてはすべての周波数の音が均等に含まれている白色の周波数の性質からである。このときの周波数からなる色の名前を用いてピンクガウス雑音などがある。サンプリング周波数が変化すると音の高さが多少低くなっている事がわかる。これはコンピュータが出たを読み出してアナログ信号に変換する速度が遅くなったためである。この結果から、デジタル音声の再生には、録音時と再生時に用いるサンプリング周波数は同値である必要がある [1]。さらに、周波数が時間とともに 440 Hz から 880 Hz へ線形に変化するチャープ波を生成した結果から、純音のような定常信号とは異なり、非定常信号では時間経過に伴って周波数成分が連続的に遷移する特性が確認できる。また、うなりの差周波数を 2 Hz、-2 Hz、10 Hz と変化させた検証からは、波形の物理的な周波数差と人間の聴覚特性の関係が考察できる。差が 2 Hz と -2 Hz の場合、うなりの周期を決定する包絡線の変動は合成波の数式における周波数差の絶対値 $|f_1 - f_2|$ に依存するため、符号に関わらずどちらも 1 秒間に 2 回の周期で同一のうなりとして観測される。一方、差を 10 Hz に広げると、振幅の変動が速くなりすぎ、人間の耳には周期的なうなりというよりも濁った粗い音として知覚されるようになる。これは、人間の聴覚が持つ時間的な振幅変動の分解能の限界を示す結果である。加えて、三角波やノコギリ波を正弦波の加算合成によって生成した結果から、波形の形状とそこに含まれる高調波成分の構成との直接的な関係性が明らかになった。ノコギリ波はすべての整数倍の高調波成分を含み振幅が $1/k$ で減衰するのに対し、三角波は奇数倍の高調波のみを含み振幅が $1/k^2$ でより急激に減衰する。このように、構成される高調波の周波数と振幅の減衰率が変化することで、時間領域での波形の形状が変化すると同時に、聴覚上で知覚される音色が明確

に異なることが確認できる。

2.6 音のフーリエ解析

フーリエ変換の結果、正弦波ではその周波数についてインパルスが出現していることがわかる。矩形はそうでなく、インパルスが規則的に出現しその強度が徐々に弱くなっていることがわかる。次の課題では High Pass Filter(HPF) をもちいてそれについてその周波数を確認する。今実験では A を 440Hz としたときの C_{maj} スケールについて F 以上の純音を省くようなフィルターを作成した。その証拠に図には F より周波数の高い純音のみしかインパルスが出現していないことがわかる。フィルターは周波数領域のみでなく時間領域でも等価であるため、時間領域でも音が削られている事がわかる。

3 画像のフーリエ解析

方法

本実験では、MATLAB を用いて画像の工学的特徴について学習し、複数の画像処理および解析を実施した。各課題の具体的な手法を以下に示す。なお、本節の課題を実装した MATLAB プログラムについては、付録 A.3 に示す。

3.1 画像の理解

3.1.1 課題 1：カラー画像のチャンネル分解と入れ替え

画像 kut.jpg を読み込み、赤、緑、青の各色チャンネル成分を抽出してグレースケール画像としてそれぞれ描画した。さらに、元の画像の配列のうち、赤色チャンネルと青色チャンネルを入れ替えた新たなカラー画像を生成した。

3.1.2 課題 2：グレースケール化と量子化

カラー画像を以下の式を用いて 8 bit のグレースケール画像に変換した。

$$Y = 0.3R + 0.59G + 0.11B$$

その後、生成した 8 bit のグレースケール画像に対してビットシフト演算 (bitshift) を行い、4 bit, 2 bit, 1 bit へ量子化ビット数を減らした画像を生成した。

3.1.3 課題 3：階調反転

課題 2 で作成した 8 bit, 4 bit, 2 bit, 1 bit の各グレースケール画像に対し、それぞれのビット数における最大階調値（それぞれ 255, 15, 3, 1）から現在の画素値を減算することで、階調反転処理を行った。

3.1.4 課題 4：閾値処理による 2 値化

8 bit のグレースケール画像に対し、閾値を 100, 127, 200 の 3 パターンに設定して 2 値化処理を行った。各閾値以上の画素値を 255（白）、それ未満の画素値を 0（黒）に変換した。

3.1.5 課題 5：ヒストグラムの算出

8 bit のグレースケール画像に含まれる 0 から 255 までの全階調値について、それぞれの値を持つ画素数を数え上げ、画像の濃度ヒストグラムを作成した。

3.1.6 課題 6：背景差分

2 枚の画像（prepicture2.jpg および postpicture2.jpg）を読み込み、それぞれ課題 2 と同じ演算式を用いてグレースケール画像に変換した。その後、2 つの画像の対応する画素ごとの差分の絶対値を計算し、背景差分画像を生成した。

3.2 画像のフィルタ処理

本実験では、MATLAB を用いて画像の工学的特徴について学習し、ノイズの付加や空間フィルタリング、色空間の変換を利用した画像解析を実施した。各課題の具体的な手法を以下に示す。なお、本節の課題を実装した MATLAB プログラムについては、付録 A.4 に示す。

3.2.1 課題 1：ノイズの付加

カラー画像をグレースケール化した後、2 種類のノイズを付加した画像を生成した。インパルスノイズは、一様乱数を用いて全体の 1% の画素値を 0（黒）、別の 1% の画素値を 255（白）に置換することで生成した。また、ガウスノイズは、標準偏差 10 の正規乱数（ガウス分布）を元の画素値に加算することで生成した。

3.2.2 課題 2：平滑化フィルタとメディアンフィルタの適用

課題 1 で生成した 2 種類のノイズ画像に対し、ノイズ除去を目的とした空間フィルタリングを適用した。平滑化フィルタとして、要素がすべて $1/9$ の 3×3 のカーネルを用いて畳み込み演算を行った。また、メディアンフィルタとして、対象画素を中心とする 3×3 の近傍領域の中央値を新たな画素値とする処理を、ゼロパディングを施した上で各画素に対して適用した。

3.2.3 課題 3：Sobel フィルタによるエッジ抽出

グレースケール画像に対し、水平方向および垂直方向のエッジを抽出するため、以下の 3×3 の Sobel フィルタをそれぞれ適用した。

$$h = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad v = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

フィルタ適用後の値は絶対値を取り，最大値が 255 となるように正規化して画像として出力した．

3.2.4 課題 4：ラプラシアンフィルタとコントラスト強調

グレースケール画像に対し，以下の 3×3 のラプラシアンフィルタを適用して 2 次微分によるエッジ抽出を行った．

$$L = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

フィルタ適用後の絶対値を 0 から 255 の範囲に正規化した後，画素値の分布における下側 5 パーセントイルおよび上側 95 パーセントイルを閾値として算出した．これらの閾値を利用して画素値の線形変換（コントラスト強調）を実施し，ヒストグラムとともに結果を描画した．

3.2.5 課題 5：HSV 色空間を用いた肌色抽出

新たな画像 (so_takahashi.jpg) を読み込み，rgb2hsv 関数を用いて RGB 色空間から HSV 色空間へ変換した．その後，色相 (Hue)，彩度 (Saturation)，明度 (Value) の各チャネルに対して特定の閾値範囲 (H: 0.022～0.087, S: 0.301～1.000, V: 0.283～0.937) を設定して論理積をとることによって肌色領域のマスクを作成し，抽出された領域のみを再度 RGB 画像として復元して出力した．

3.3 画像のフーリエ変換・逆フーリエ変換

本実験では，MATLAB を用いて画像の 2 次元フーリエ変換および逆フーリエ変換を利用した周波数解析とフィルタリング処理を実施した．各課題の具体的な手法を以下に示す．なお，本節の課題を実装した MATLAB プログラムについては，付録 A.5 に示す．

3.3.1 課題 1：垂直方向の正弦波画像の周波数解析

サイズ 256×256 ピクセルの画像に対し，空間周波数 f を 4, 16, 64 と変化させた垂直方向の正弦波（縦縞）を描画した．生成した画像に対し，fft2 関数を用いて 2 次元高速フーリエ変換を行い，fftshift 関数で低周波成分が中心にくるようにシフトさせた後，振幅の 2 乗を計算してパワースペクトルを算出した．元の画像，全体のパワースペクトル，および中心付近を拡大したパワースペクトルを描画した．

3.3.2 課題 2：水平方向の正弦波画像の周波数解析

課題 1 と同様にサイズ 256×256 ピクセルの画像に対し，空間周波数 f を 4, 16, 64 と変化させた水平方向の正弦波（横縞）を描画した．生成した画像に対して 2 次元フーリエ変換およびシフト処理を行い，パワースペクトルを算出して画像とともに描画した．

3.3.3 課題 3：矩形画像の平行移動とパワースペクトル

サイズ 256×256 ピクセルの背景（画素値 0）に対し、サイズ 31×61 ピクセルの矩形（画素値 255）を配置した画像を生成した。矩形の位置を画像中心に配置した場合と、縦横にそれぞれ 60 ピクセル平行移動させた場合の 2 パターンについて、2 次元フーリエ変換を行ってパワースペクトルを算出した。

3.3.4 課題 4：ローパスフィルタと逆フーリエ変換

画像（lena_gray.bmp）を読み込み、2 次元フーリエ変換およびシフト処理を行って周波数領域のデータを取得した。画像中心からの距離 D を算出し、指定したカットオフ半径 R 以下の成分を 1、それ以外を 0 とする円形のローパスフィルタを作成した。半径 R は、無限大（フィルタなし）、10、50 の 3 パターンに設定した。周波数データにフィルタを乗算して高周波成分を遮断した後、`ifftshift` 関数および `ifft2` 関数を用いて逆フーリエ変換を適用し、空間領域の画像へ復元して結果を描画した。

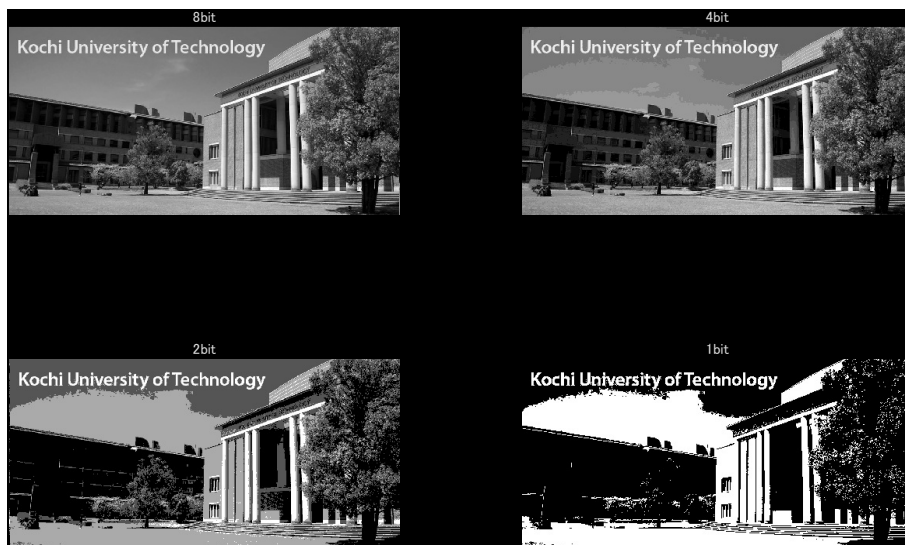
結果

3.3.5 画像の理解

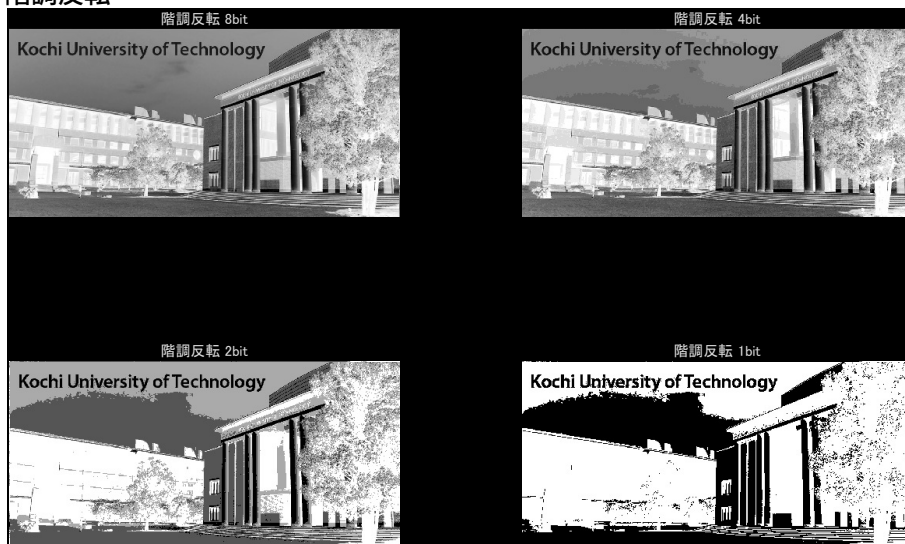
課題 1：カラー画像のチャンネル分解と入れ替え



課題 2：グレースケール化と量子化



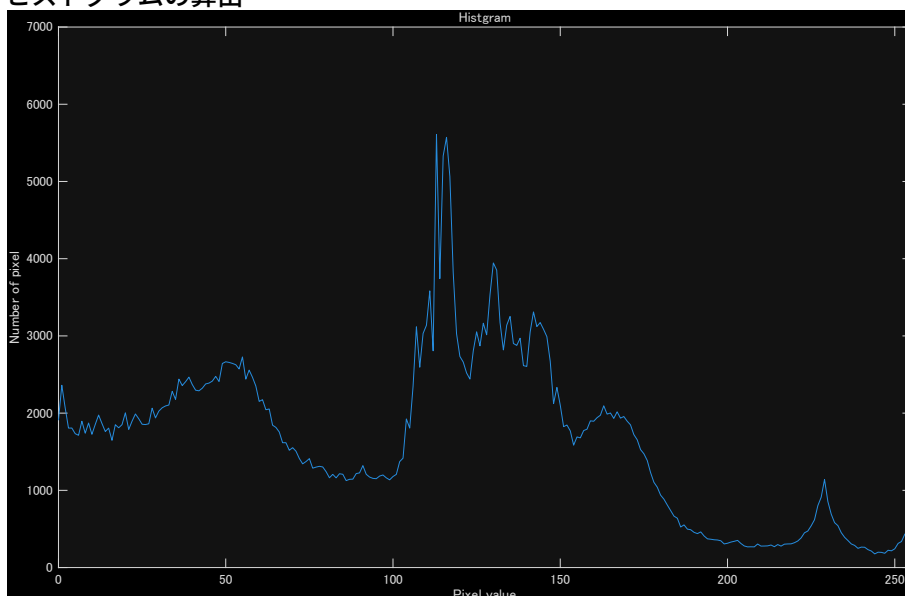
課題 3：階調反転



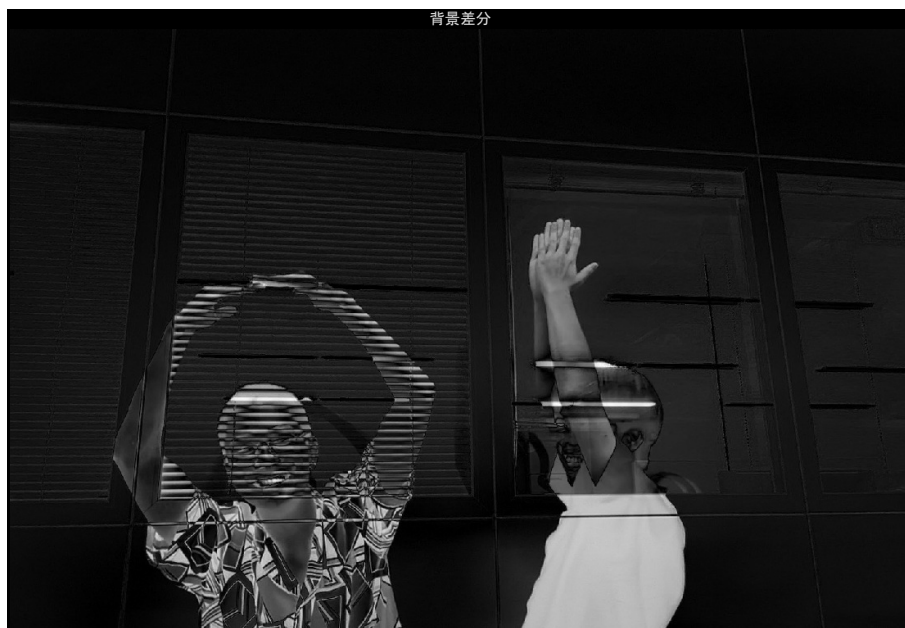
課題 4：閾値処理による 2 値化



課題 5：ヒストグラムの算出

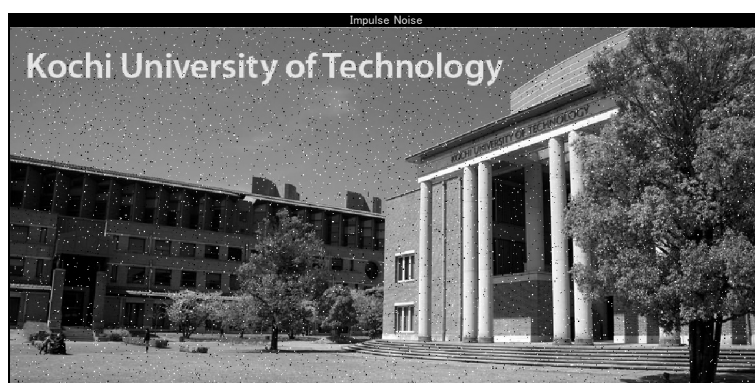


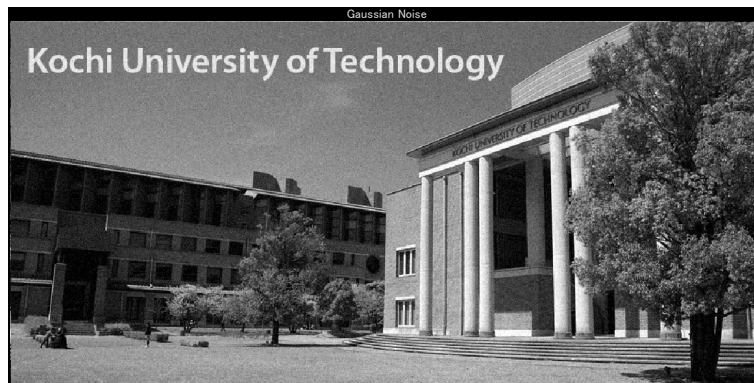
課題 6：背景差分



3.3.6 画像のフィルタ処理

課題 1：ノイズの付加





課題 2：平滑化フィルタとメディアンフィルタの適用





課題 3：Sobel フィルタによるエッジ抽出



課題 4：ラプラシアンフィルタとコントラスト強調

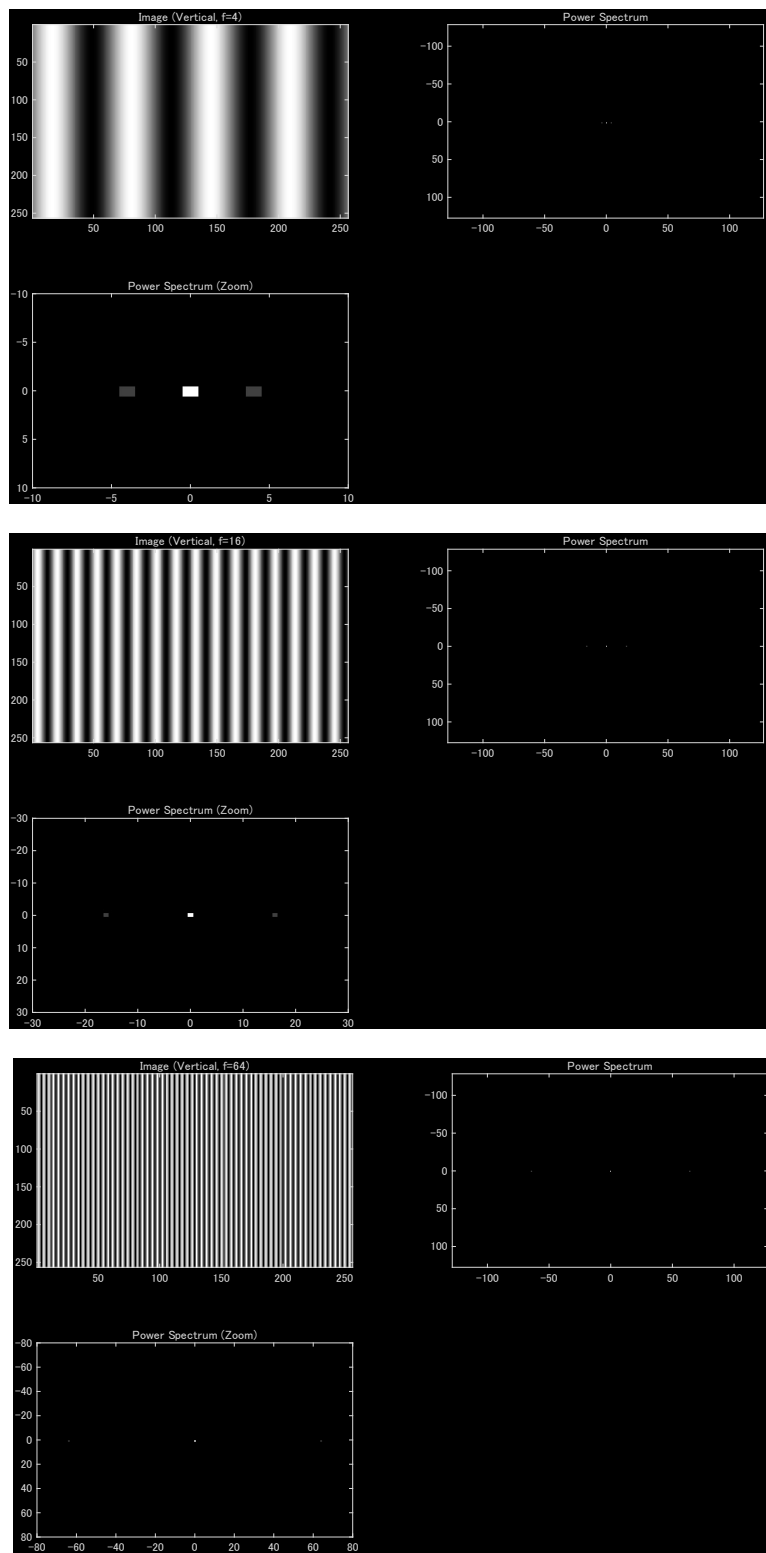


課題 5：HSV 色空間を用いた肌色抽出

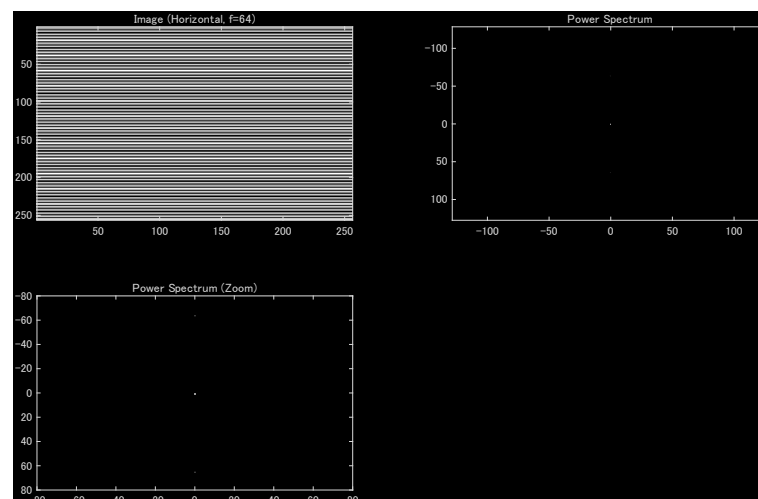
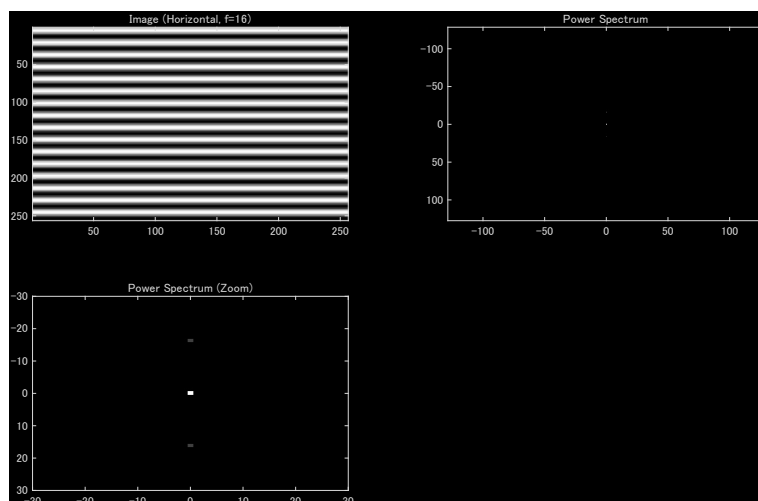
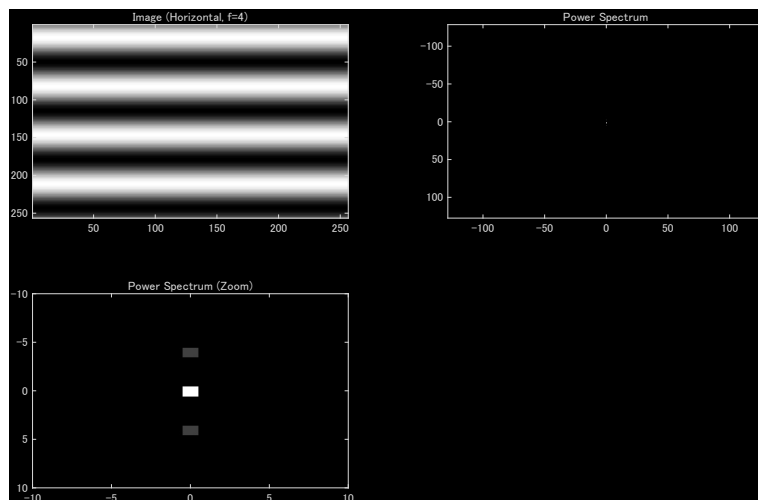


3.3.7 画像のフーリエ変換・逆フーリエ変換

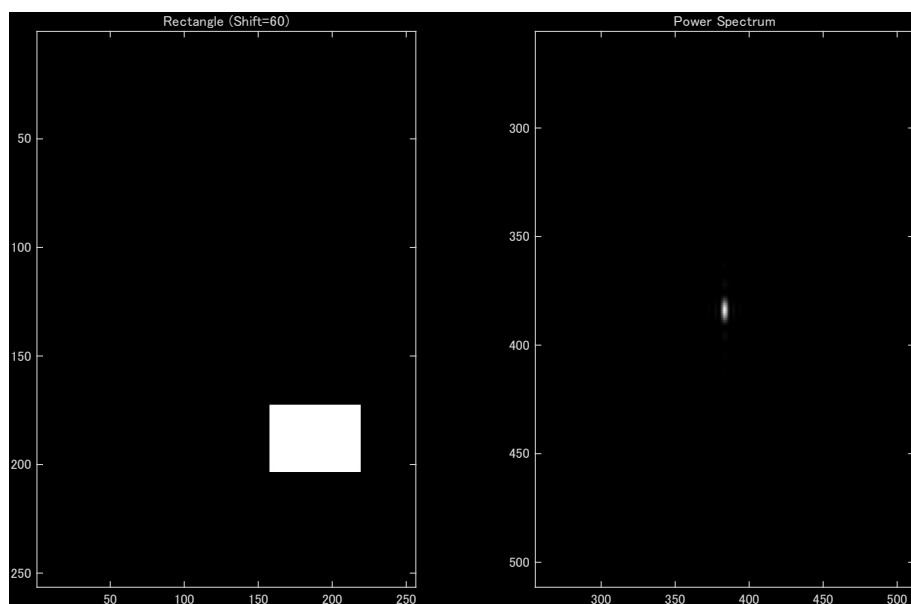
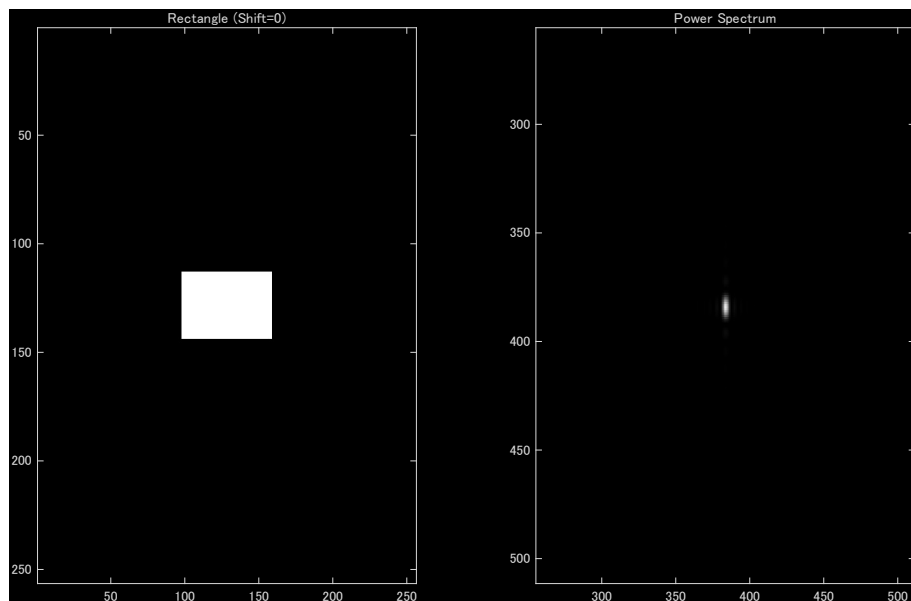
課題 1：垂直方向の正弦波画像の周波数解析



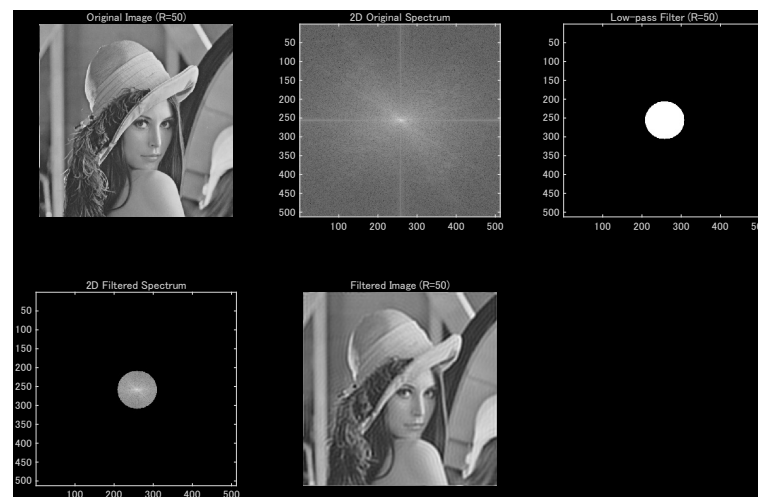
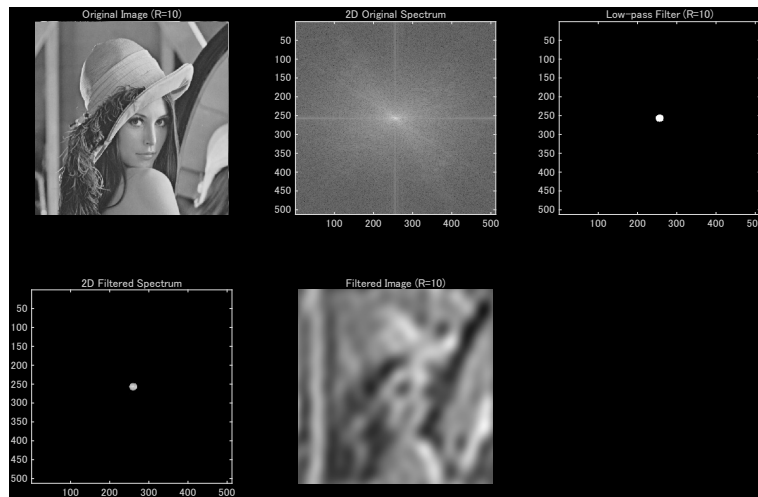
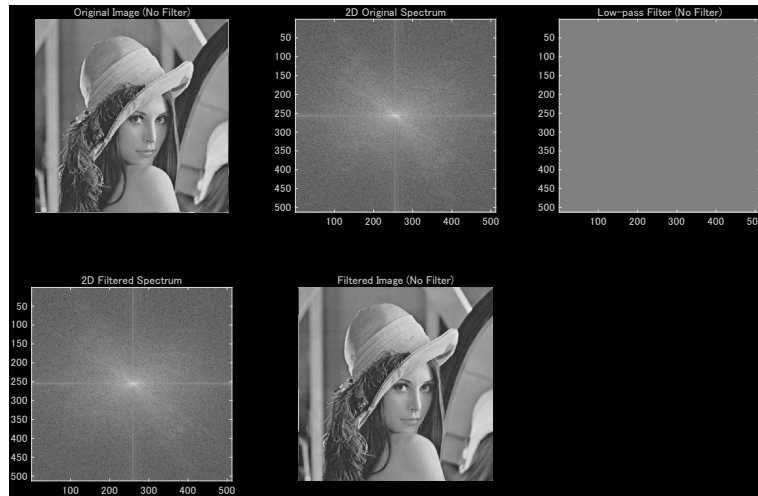
課題 2：水平方向の正弦波画像の周波数解析



課題 3：矩形画像の平行移動とパワースペクトル



課題 4：ローパスフィルタと逆フーリエ変換



考察

3.4 画像の理解

画像のチャンネル分解と入れ替えの結果から、カラー画像は赤、緑、青（RGB）の3つの独立した成分の加法混色によって表現されていることがわかる。グレースケール化と量子化について、8 bit から 4 bit, 2 bit, 1 bit へと表現できる階調数を減らすにつれて、画像に擬似輪郭が発生し、滑らかな濃淡が失われていくことが確認できる。これは画素が取り得る値の解像度が低下したためである。階調反転や閾値処理（2 値化）は、画素値に対する線形または非線形の写像関数を適用する操作と同値である。また、背景差分では、2 枚の画像間の対応する画素の差分絶対値をとることで、背景が相殺され、変化した領域（動体など）のみが抽出されることがわかる。これは監視カメラなどの動体検知の基礎的な原理と言える。

3.5 画像のフィルタ処理

ノイズの性質と空間フィルタの比較から、ノイズの種類に応じた適切なフィルタ選択の重要性が考察できる。インパルスノイズに対しては、極端な画素値（外れ値）の影響を受けにくいメディアンフィルタ（中央値フィルタ）が、エッジのぼやけを抑えつつ効果的にノイズを除去できることがわかる [2]。一方、ガウス分布に従うノイズに対しては平滑化フィルタ（平均化）が一定のノイズ低減効果を示す。エッジ抽出において、Sobel フィルタは 1 次微分に基づくため、水平または垂直など特定の方向の勾配に強く反応するのに対し、ラプラシアンフィルタは 2 次微分に基づくため全方向の輪郭を抽出できる。ただし、2 次微分は微小なノイズに対しても敏感に反応して値を増幅させるため、実用上は事前の平滑化が必要となる場合が多い。さらに、HSV 色空間を用いた肌色抽出では、RGB 色空間とは異なり色相（H）、彩度（S）、明度（V）が分離されているため、照明の明るさの変化（明度）に影響されにくく、特定の対象物をロバスト（頑健）に抽出できることがわかる [3]。

3.6 画像のフーリエ変換・逆フーリエ変換

画像の 2 次元フーリエ変換の結果から、空間領域における周期的なパターンの細かさ（空間周波数）と方向が、周波数領域（パワースペクトル）におけるインパルスの位置として直感的に表現されることがわかる [4]。このとき、ゼロ周波数成分（直流成分）を配列の中心にシフトさせることで、低周波から高周波への対称的な広がりが見えやすくなる [5]。縦縞（水平方向に画素値が変動）の場合は水平軸上に、横縞（垂直方向に変動）の場合は垂直軸上にインパルスが現れ、空間周波数 f が高いほど中心（直流成分）から遠い位置にピークが出現している。また、矩形画像を空間領域で平行移動させても、パワースペクトルの形状には変化が見られない。これは、フーリエ変換の推移定理（シフト定理）により、空間領域での平行移動は周波数領域における位相の回転のみをもたらす、振幅成分（およびその 2 乗であるパワースペクトル）には影響を与えないためである。最後に、ローパスフィルタを用いた逆フーリエ変換の結果から、カットオフ半径 R

を小さくして高周波成分を多く遮断するほど、復元された画像全体がぼやけることが確認できる。このことから、画像における輪郭や細かなディテール（エッジ領域）は高周波数成分によって構成されており、平坦な領域は低周波数成分で構成されていると言える。

4 結論

本実験では、MATLAB を用いて音響信号および画像データに対する多様な処理と解析を行い、物理現象をデジタルデータとして工学的に扱うための基礎理論とその応用手法について学習した。

音の信号解析においては、正弦波の生成や加算合成を通じて、振幅、周波数、位相といった波形の基本パラメータが聴覚に与える影響や、サンプリング定理に基づくデジタル化の重要性を確認した。また、フーリエ変換およびフィルタ処理により、時間領域の信号を周波数領域に変換して成分を評価し、特定の周波数帯域を制御・抽出する手法を習得した。

画像解析においては、画素の階調操作や色空間の変換といった基礎的な空間領域の処理をはじめ、空間フィルタリングによるノイズ除去やエッジ抽出の特性を比較・検証した。さらに、画像の2次元フーリエ変換を通じて空間周波数の概念を視覚的に理解し、周波数領域におけるローパスフィルタが画像の平滑化に寄与することを明らかにした。

総じて、音と画像という一見異なる物理メディアであっても、デジタル信号処理とフーリエ解析という共通の数学的・情報学的基盤を用いてその特性を統一的に記述・操作できることが確認できた。本実験を通じて、これらの技術が高度な情報通信システムやパターン認識技術の不可欠な土台となっていることを深く理解することができた。

参考文献

- [1] MathWorks, “sound -オーディオとして再生する”,
<https://jp.mathworks.com/help/matlab/ref/sound.html> (参照 2026-08-09).
- [2] MathWorks, “medfilt2 - 2次元メディアンフィルター”,
<https://jp.mathworks.com/help/images/ref/medfilt2.html> (参照 2026-08-11).
- [3] MathWorks, “rgb2hsv - RGBカラー値をHSVカラー空間に変換”,
<https://jp.mathworks.com/help/matlab/ref/rgb2hsv.html> (参照 2026-08-11).
- [4] MathWorks, “fft2 - 2次元高速フーリエ変換”,
<https://jp.mathworks.com/help/matlab/ref/fft2.html> (参照 2026-08-11).
- [5] MathWorks, “fftshift - ゼロ周波数成分を配列の中心にシフト”,
<https://jp.mathworks.com/help/matlab/ref/fftshift.html> (参照 2026-08-11).

A MATLAB プログラムコード

A.1 音の信号生成と解析に関する実装

Listing 1 class1/exp4_3_1.m

```
1 clear all;
2
3 Fs = 16000;
4
5 % 課題1(440Hz) %
6 t = 0 : 1 / Fs : 1;
7
8 y1_1 = sin(2*pi*440*t);
9 sound(y1_1, Fs);
10 pause(2);
11
12 subplot(2, 1, 1);
13 plot(t(1:220), y1_1(1:220));
14 title('$\sin(2 \pi \times 440 t)$', 'Interpreter', 'latex', 'FontSize', 14);
15
16 % 課題1(660Hz) %
17 y1_2 = sin(2*pi*660*t);
18 sound(y1_2, Fs);
19 pause(2);
20
21 subplot(2, 1, 2);
22 plot(t(1:220), y1_2(1:220));
23 title('$\sin(2 \pi \times 660 t)$', 'Interpreter', 'latex', 'FontSize', 14);
24
25 figure;
26
27 % 課題2 %
28 y2_1 = 0.5*sin(2*pi*440*t + 1 / 2*pi);
```

```

29 y2_2 = 0.5*sin(2*pi*440*t + pi);
30 y2_3 = 0.25*sin(2*pi*440*t + 1 / 2*pi);
31 y2_4 = 0.25*sin(2*pi*440*t + pi);
32
33 sound(y2_1, Fs);
34 pause(2);
35 sound(y2_2, Fs);
36 pause(2);
37 sound(y2_3, Fs);
38 pause(2);
39 sound(y2_4, Fs);
40 pause(2);
41
42 subplot(2,2,1);
43 plot(t(1:220), y2_1(1:220));
44 title('$$ 0.5 \cdot \sin(2\pi \cdot 440t + \frac{\pi}{2}) $$', '
    Interpreter', 'latex', 'FontSize', 14);
45
46 subplot(2,2,2);
47 plot(t(1:220), y2_2(1:220));
48 title('$$ 0.5 \sin(2\pi \cdot 440 t + \pi) $$', 'Interpreter', 'latex', '
    FontSize', 12);
49
50 subplot(2,2,3);
51 plot(t(1:220), y2_3(1:220));
52 title('$$ 0.25 \sin\left(2\pi \cdot 440 t + \frac{\pi}{2}\right) $$', '
    Interpreter', 'latex', 'FontSize', 12);
53
54 subplot(2,2,4);
55 plot(t(1:220), y2_4(1:220));
56 title('$$ 0.25 \sin(2\pi \cdot 440 t + \pi) $$', 'Interpreter', 'latex',
    'FontSize', 12);
57 figure;
58
59 % 課題3 %
60 t = 0 : 1 / Fs : 4;

```

```

61 y3 = sin(2 * pi * 440 * t) + sin(2 * pi * 441 * t);
62 sound(y3, Fs);
63 pause(2);
64
65 plot(t, y3);
66 title('$$ \sin(2\pi \cdot 440 t) + \sin(2\pi \cdot 441 t) $$', '
    Interpreter', 'latex', 'FontSize', 12);
67
68 % 課題4 %
69 t = 0 : 1 / Fs : 1;
70
71 N_values = [1, 5, 25];
72 figure;
73
74 for i = 1:length(N_values)
75     N = N_values(i);
76     y_sum = zeros(size(t));
77
78     for k = 1:N
79         y_sum = y_sum + (1 / (2*k - 1)) * sin(2*pi*(2*k - 1)*220*t);
80     end
81     sound(y_sum, Fs);
82     pause(2);
83
84     subplot(3, 1, i);
85     plot(t(1:220), y_sum(1:220));
86
87     title_str = ['$$ f(t) = \sum_{k=1}^{' , num2str(N), '} \frac{1}{2k -
        1} \sin(2\pi(2k - 1)ft) $$'];
88     title(title_str, 'Interpreter', 'latex', 'FontSize', 12);
89 end
90
91 % 課題5 %
92 Fs = 16000;
93 t = 0:1 / Fs:1;
94

```

```

95 noise = 0.2 * randn(1, length(t));
96 sound(noise, Fs);
97 pause(2);
98
99 figure;
100
101 subplot(2, 1, 1);
102 plot(t, noise);
103 title('白色ガウス雑音 (\mu=0, \sigma=0.2)');
104 grid on;
105
106 [noise, c] = hist(noise, 50);
107 subplot(2, 1, 2);
108 plot(c, noise);
109
110 % 発展課題 %
111 Fs = 15000;
112 t = 0:1 / Fs:1;
113
114 y_a1 = sin(2 * pi * 440 * t);
115 sound(y_a1, Fs);
116 pause(1.5);
117
118 figure;
119
120 subplot(2, 1, 1);
121 plot(t(1:220), y1_1(1:220));
122 title('$y(t) \quad (F_s = 16000\mathrm{Hz})$', 'Interpreter', 'latex', '
    FontSize', 12);
123
124 subplot(2, 1, 2);
125 plot(t(1:220), y_a1(1:220));
126 title('$y(t) \quad (F_s = 15000\mathrm{Hz})$', 'Interpreter', 'latex', '
    FontSize', 12);
127
128 % 発展課題 %

```

```

129 Fs = 8000;
130 t = 0 : 1/Fs : 0.5;
131
132 y = sin(2 * pi * (440 * t + 440 * t.^2));
133
134 sound(y, Fs);
135 pause(1);
136
137 figure;
138
139 subplot(2, 1, 1);
140 idx_start = 1 : round(0.02 * Fs);
141 plot(t(idx_start), y(idx_start));
142 title('$y(t)$ at Start (Around 440Hz)', 'Interpreter', 'latex', 'FontSize', 12);
143
144 subplot(2, 1, 2);
145 idx_end = length(t) - round(0.02 * Fs) : length(t);
146 plot(t(idx_end), y(idx_end));
147 title('$y(t)$ at End (Around 880Hz)', 'Interpreter', 'latex', 'FontSize', 12);
148 grid on;
149
150 % 発展課題 %
151
152 Fs = 16000;
153 t = 0 : 1/Fs : 2;
154
155 y_a3_1 = sin(2 * pi * 440 * t) + sin(2 * pi * 442 * t);
156 y_a3_2 = sin(2 * pi * 440 * t) + sin(2 * pi * 438 * t);
157 y_a3_3 = sin(2 * pi * 440 * t) + sin(2 * pi * 450 * t);
158
159 soundsc(y_a3_1, Fs);
160 pause(2.5);
161
162 soundsc(y_a3_2, Fs);

```

```

163 pause(2.5);
164
165 soundsc(y_a3_3, Fs);
166 pause(2.5);
167
168 figure;
169 idx_1sec = 1 : Fs;
170
171 subplot(3, 1, 1);
172 plot(t(idx_1sec), y_a3_1(idx_1sec));
173 title('$$ 440\mathrm{Hz} + 442\mathrm{Hz} \quad (\Delta f = 2\mathrm{Hz}) $$', 'Interpreter', 'latex', 'FontSize', 12);
174 grid on;
175
176 subplot(3, 1, 2);
177 plot(t(idx_1sec), y_a3_2(idx_1sec));
178 title('$$ 440\mathrm{Hz} + 438\mathrm{Hz} \quad (\Delta f = -2\mathrm{Hz} ) $$', 'Interpreter', 'latex', 'FontSize', 12);
179 grid on;
180
181 subplot(3, 1, 3);
182 plot(t(idx_1sec), y_a3_3(idx_1sec));
183 title('$$ 440\mathrm{Hz} + 450\mathrm{Hz} \quad (\Delta f = 10\mathrm{Hz} ) $$', 'Interpreter', 'latex', 'FontSize', 12);
184 grid on;
185
186 % 発展課題 %
187
188 Fs = 16000;
189 f = 2;
190 t = 0 : 1/Fs : 1.5;
191
192 N_values = [1, 5, 25];
193 figure;
194
195 for i = 1:length(N_values)

```

```

196     N = N_values(i);
197     y_tri = zeros(size(t));
198
199     for k = 1:N
200         y_tri = y_tri + ((-1)^(k-1) / (2*k - 1)^2) * sin(2 * pi * (2*k -
                1) * f * t);
201     end
202
203     subplot(2, 3, i);
204     plot(t, y_tri);
205     title(['$$ \mathrm{Triangle} \quad (N=', num2str(N), ') $$'], '
            Interpreter', 'latex', 'FontSize', 12);
206     grid on;
207 end
208
209 for i = 1:length(N_values)
210     N = N_values(i);
211     y_saw = zeros(size(t));
212
213     for k = 1:N
214         y_saw = y_saw + ((-1)^(k-1) / k) * sin(2 * pi * k * f * t);
215     end
216
217     subplot(2, 3, i + 3);
218     plot(t, y_saw);
219     title(['$$ \mathrm{Sawtooth} \quad (N=', num2str(N), ') $$'], '
            Interpreter', 'latex', 'FontSize', 12);
220     grid on;
221 end

```

A.2 音のフーリエ解析とフィルタ処理に関する実装

Listing 2 class2/exp4_3_2.m

```

1 clear all;
2 % サンプリング周波数から時間軸の作成
3 Fs = 8192;
4 t = 0:1 / Fs:1;

```

```

5
6 %% 課題1 %%
7 % 周波数440HzよりAの音を作成
8 f = 440;
9 signal_A = sin(2 * pi * f * t);
10 signal_B = (1:1024);
11 for k = signal_B
12     if mod(floor(k / 129), 2) == 0
13         signal_B(k) = 0;
14     else
15         signal_B(k) = 1;
16     end
17 end
18 fft_signal_A = fft(signal_A);
19 fft_signal_B = fft(signal_B);
20 fft_shift_A = fftshift(fft_signal_A);
21 fft_shift_B = fftshift(fft_signal_B);
22
23 figure(1);
24 T_A = length(signal_A) / Fs;
25 faxis_A = [-Fs/2: 1/T_A: Fs/2-1/T_A];
26 subplot(1, 2, 1);
27 plot(faxis_A, abs(fft_shift_A));
28 xlim([0 4500]);
29 title('振幅スペクトル：正弦波 (440Hz)'); % 追記：タイトル
30 xlabel('Frequency [Hz]'); % 追記：横軸ラベル
31 ylabel('Amplitude'); % 追記：縦軸ラベル
32 grid on; % 追記：グリッド線
33
34 T_B = length(signal_B) / Fs;
35 faxis_B = [-Fs/2: 1/T_B: Fs/2-1/T_B];
36 subplot(1, 2, 2);
37 plot(faxis_B, abs(fft_shift_B));
38 xlim([0 200]);
39 title('振幅スペクトル：矩形波'); % 追記：タイトル
40 xlabel('Frequency [Hz]'); % 追記：横軸ラベル

```

```

41 ylabel('Amplitude'); % 追記：縦軸ラベル
42 grid on; % 追記：グリッド線
43
44 %% 課題2 %%
45 t = 0: 1/Fs : 0.25 - 1/Fs;
46 C = sin(2 * pi * 261.38 * t);
47 D = sin(2 * pi * 293.67 * t);
48 E = sin(2 * pi * 329.63 * t);
49 F = sin(2 * pi * 349.23 * t);
50 G = sin(2 * pi * 392.00 * t);
51 A = sin(2 * pi * 440.00 * t);
52 B = sin(2 * pi * 493.88 * t);
53 hi_C = sin(2 * pi * 523.23 * t);
54 melody = [C, D, E, F, G, A, B, hi_C];
55 fft_melody = fft(melody);
56 fft_shift_melody = fftshift(fft_melody);
57 T_melody = length(melody) / Fs;
58 faxis_melody = -Fs/2: 1/T_melody: Fs/2-1/T_melody;
59
60 figure(2);
61 subplot(2,2,1);
62 plot(faxis_melody, abs(fft_shift_melody));
63 xlim([200 600]);
64 title('振幅スペクトル：原信号 (メロディ)'); % 追記：タイトル
65 xlabel('Frequency [Hz]'); % 追記：横軸ラベル
66 ylabel('Amplitude'); % 追記：縦軸ラベル
67 grid on; % 追記：グリッド線
68
69 cut_off_sig = 357;
70 for k = 1:length(melody)
71     if faxis_melody(k) < cut_off_sig
72         fft_shift_melody(k) = 0;
73     end
74 end
75
76 subplot(2, 2, 2);

```

```

77 plot(faxis_melody, abs(fft_shift_melody));
78 xlim([200 600]);
79 title('振幅スペクトル：フィルタ適用後'); % 追記：タイトル
80 xlabel('Frequency [Hz]'); % 追記：横軸ラベル
81 ylabel('Amplitude'); % 追記：縦軸ラベル
82 grid on; % 追記：グリッド線
83
84 ifft_shift_melody = ifftshift(fft_shift_melody);
85 ifft_melody = ifft(ifft_shift_melody);
86 t_total = (0 : length(ifft_melody) - 1) / Fs;
87
88 subplot(2, 2, 3);
89 plot(t_total, real(ifft_melody));
90 title('時間波形：フィルタ適用後'); % 追記：タイトル
91 xlabel('Time [s]'); % 追記：横軸ラベル
92 ylabel('Amplitude'); % 追記：縦軸ラベル
93 grid on; % 追記：グリッド線

```

A.3 画像と色調

Listing 3 class3/exp4_3_3.m

```

1 clear;
2
3 % 課題1 %
4 img = imread("kut.jpg");
5
6 red_gray = img(:,:,1);
7 green_gray = img(:,:,2);
8 blue_gray = img(:,:,3);
9
10 figure(1)
11 subplot(2,2,1);
12 imshow(uint8(red_gray));
13 title("赤色チャネル");
14 subplot(2,2,2);
15 imshow(uint8(green_gray));
16 title("緑色チャネル");

```

```

17 subplot(2,2,3);
18 imshow(uint8(blue_gray));
19 title("青色チャネル");
20
21 [height, width, ch] = size(img);
22 img_exchange_red_and_blue = zeros(height, width, ch);
23 img_exchange_red_and_blue(:,:,1) = img(:,:,3);
24 img_exchange_red_and_blue(:,:,2) = img(:,:,2);
25 img_exchange_red_and_blue(:,:,3) = img(:,:,1);
26
27 subplot(2,2,4);
28 imshow(uint8(img_exchange_red_and_blue));
29 title("赤色チャネルと青色チャネル入れ替え");
30
31
32 % 課題2 %
33 gimg = 0.3 * double(img(:,:,1)) + 0.59 * double(img(:,:,2)) + 0.11 *
    double(img(:,:,3));
34
35 bit8_gimg = uint8(gimg);
36 bit4_gimg = bitshift(uint8(gimg), -4);
37 bit2_gimg = bitshift(uint8(gimg), -6);
38 bit1_gimg = bitshift(uint8(gimg), -7);
39
40 figure(2)
41 subplot(2,2,1);
42 imshow(bit8_gimg, [0, 255]);
43 title("8bit");
44 subplot(2,2,2);
45 imshow(bit4_gimg, [0, 15]);
46 title("4bit");
47 subplot(2,2,3);
48 imshow(bit2_gimg, [0, 3]);
49 title("2bit");
50 subplot(2,2,4);
51 imshow(bit1_gimg, [0, 1]);

```

```

52 title("1bit");
53
54 imwrite(uint8(bit8_gimg), "kut_bit8.png");
55 imwrite(uint8(bit4_gimg * (255 / 15)), "kut_bit4.png");
56 imwrite(uint8(bit2_gimg * (255 / 3)), "kut_bit2.png");
57 imwrite(uint8(bit1_gimg * (255 / 1)), "kut_bit1.png");
58
59
60 % 課題3 %
61 neg_bit8_gimg = 255 - bit8_gimg;
62 neg_bit4_gimg = 15 - bit4_gimg;
63 neg_bit2_gimg = 3 - bit2_gimg;
64 neg_bit1_gimg = 1 - bit1_gimg;
65
66 figure(3)
67 subplot(2,2,1);
68 imshow(neg_bit8_gimg, [0, 255]);
69 title("階調反転 8bit");
70 subplot(2,2,2);
71 imshow(neg_bit4_gimg, [0, 15]);
72 title("階調反転 4bit");
73 subplot(2,2,3);
74 imshow(neg_bit2_gimg, [0, 3]);
75 title("階調反転 2bit");
76 subplot(2,2,4);
77 imshow(neg_bit1_gimg, [0, 1]);
78 title("階調反転 1bit");
79
80 imwrite(uint8(neg_bit8_gimg), "kut_neg_bit8.png");
81 imwrite(uint8(neg_bit4_gimg * (255 / 15)), "kut_neg_bit4.png");
82 imwrite(uint8(neg_bit2_gimg * (255 / 3)), "kut_neg_bit2.png");
83 imwrite(uint8(neg_bit1_gimg * (255 / 1)), "kut_neg_bit1.png");
84
85
86 % 課題4 %
87 thr100 = uint8(gimg >= 100);

```

```

88 thr127 = uint8(gimg >= 127);
89 thr200 = uint8(gimg >= 200);
90
91 gimg100 = thr100 * 255;
92 gimg127 = thr127 * 255;
93 gimg200 = thr200 * 255;
94
95 figure(4)
96 subplot(2,2,1);
97 imshow(uint8(gimg));
98 title("元画像(グレースケール)");
99 subplot(2,2,2);
100 imshow(gimg100);
101 title("閾値100");
102 subplot(2,2,3);
103 imshow(gimg127);
104 title("閾値127");
105 subplot(2,2,4);
106 imshow(gimg200);
107 title("閾値200");
108
109
110 % 課題5 %
111 inhist = zeros(1,256);
112 for k = 0:255
113     inhist(1, k+1) = sum(sum(uint8(gimg) == k));
114 end
115
116 figure(5)
117 plot((0:255), inhist);
118 axis([0 255 0 7000]);
119 xlabel('Pixel value');
120 ylabel('Number of pixel');
121 title('Histogram');
122
123

```

```

124 % 課題6 %
125 pre_img = imread('prepicture2.jpg');
126 post_img = imread('postpicture2.jpg');
127
128 pre_gray = uint8(0.3 * double(pre_img(:,:,1)) + 0.59 * double(pre_img
    (:,:,2)) + 0.11 * double(pre_img(:,:,3)));
129 post_gray = uint8(0.3 * double(post_img(:,:,1)) + 0.59 * double(post_img
    (:,:,2)) + 0.11 * double(post_img(:,:,3)));
130
131 sub_img = abs(double(pre_gray) - double(post_gray));
132
133 figure(6)
134 imshow(uint8(sub_img));
135 title("背景差分");
136 imwrite(uint8(sub_img), 'difference.png');

```

A.4 画像とフィルタ

Listing 4 class4/exp4_3_4.m

```

1 %% 課題1
2 clear;
3 img = imread("kut.jpg");
4 gray = 0.3*double(img(:,:,1)) + 0.59*double(img(:,:,2)) + 0.11*double(img
    (:,:,3));
5 gray = uint8(gray);
6 [height, width] = size(gray);
7
8 rd_impluse = rand(height, width);
9 img_impluse = gray;
10 img_impluse(rd_impluse <= 0.01) = 0;
11 img_impluse(rd_impluse >= 0.99) = 255;
12
13 img_gauss = uint8(double(gray) + 10 * randn(height, width));
14
15 figure(1);
16 imshow(gray); title('Original Grayscale');
17 figure(2);

```

```

18 imshow(img_impluse); title('Impulse Noise');
19 figure(3);
20 imshow(img_gauss); title('Gaussian Noise');
21
22 %% 課題2
23 smoothing_fil = ones(3) / 9;
24 img_impulse_smoothed = uint8(filter2(smoothing_fil, double(img_impluse),
    "same"));
25 img_gauss_smoothed = uint8(filter2(smoothing_fil, double(img_gauss), "
    same"));
26
27 img_impulse_padded = zeros(height+2, width+2);
28 img_gauss_padded = zeros(height+2, width+2);
29 img_impulse_padded(2:height+1, 2:width+1) = double(img_impluse(1:height,
    1:width));
30 img_gauss_padded(2:height+1, 2:width+1) = double(img_gauss(1:height, 1:
    width));
31
32 img_impulse_median = zeros(height, width);
33 img_gauss_median = zeros(height, width);
34
35 for h = 1:height
36     for w = 1:width
37         img_impulse_median(h,w) = median(img_impulse_padded(h:h+2, w:w
            +2), "all");
38         img_gauss_median(h,w) = median(img_gauss_padded(h:h+2, w:w+2), "
            all");
39     end
40 end
41
42 figure(4);
43 imshow(img_impulse_smoothed); title('Impulse Smoothed');
44 figure(5);
45 imshow(img_gauss_smoothed); title('Gauss Smoothed');
46 figure(6);
47 imshow(uint8(img_impulse_median)); title('Impulse Median');

```

```

48 figure(7);
49 imshow(uint8(img_gauss_median)); title('Gauss Median');
50
51 %% 課題3
52 sobel_h = [-1 0 1; -2 0 2; -1 0 1];
53 sobel_v = [-1 -2 -1; 0 0 0; 1 2 1];
54
55 img_sobel_h = filter2(sobel_h, double(gray), "same");
56 img_sobel_v = filter2(sobel_v, double(gray), "same");
57
58 img_sobel_h_norm = 255 * abs(img_sobel_h) / max(abs(img_sobel_h(:)));
59 img_sobel_v_norm = 255 * abs(img_sobel_v) / max(abs(img_sobel_v(:)));
60
61 figure(8);
62 subplot(1,2,1); imshow(uint8(img_sobel_h_norm)); title('Sobel H');
63 subplot(1,2,2); imshow(uint8(img_sobel_v_norm)); title('Sobel V');
64
65 %% 課題4
66 lap_fil = [1 1 1; 1 -8 1; 1 1 1];
67 gray_lap = filter2(lap_fil, double(gray), "same");
68
69 img_gray_lap_norm = 255 * abs(gray_lap) / max(abs(gray_lap(:)));
70
71 low_thr = prctile(img_gray_lap_norm, 5, "all");
72 high_thr = prctile(img_gray_lap_norm, 95, "all");
73
74 mask_low = img_gray_lap_norm <= low_thr;
75 mask_high = img_gray_lap_norm >= high_thr;
76 mask_mid = ~mask_low & ~mask_high;
77
78 img_gray_lap_norm_thr = zeros(height, width);
79 img_gray_lap_norm_thr(mask_high) = 255;
80 img_gray_lap_norm_thr(mask_mid) = 255 * (img_gray_lap_norm(mask_mid) -
    low_thr) / (high_thr - low_thr);
81
82 figure(9);

```

```

83 subplot(2,2,1);
84 imshow(uint8(img_gray_lap_norm));
85 title('Laplacian Norm');
86
87 subplot(2,2,2);
88 imshow(uint8(img_gray_lap_norm_thr));
89 title('Laplacian Threshold');
90
91 subplot(2,2,3);
92 histogram(img_gray_lap_norm, 256);
93 axis([0 255 0 150000]);
94 xlabel('pixel value');
95 ylabel('number of pixels');
96 title('Histogram');
97
98 %% 課題5
99 clear;
100 img = imread("so_takahashi.jpg");
101 img_hsv = rgb2hsv(img);
102
103 skin_mask = (img_hsv(:,:,1) >= 0.022) & (img_hsv(:,:,1) <= 0.087) & ...
104             (img_hsv(:,:,2) >= 0.301) & (img_hsv(:,:,2) <= 1.000) & ...
105             (img_hsv(:,:,3) >= 0.283) & (img_hsv(:,:,3) <= 0.937);
106
107 img_out = hsv2rgb(img_hsv .* skin_mask);
108
109 figure(10);
110 imshow(img_out); title('Skin Detection');

```

A.5 画像のフーリエ解析

Listing 5 class5/exp4_3_4.m

```

1 clear;
2 h = 256;
3 w = 256;
4
5 %% 課題1 %%

```

```

6  f = [4, 16, 64];
7  lim_vals = [10, 30, 80];
8  for f_len = 1:length(f)
9      n = 1;
10
11     for l = 1:h
12         for k = 1:w
13             img(l, k) = 1 + sin(2 * pi * f(f_len) * (k-1) / w);
14         end
15     end
16
17     fft_image = fft2(double(img));
18     fft_shift_image = fftshift(fft_image);
19     fft_shift_abs = abs(fft_shift_image);
20     ps = power(fft_shift_abs, 2);
21
22     figure(f_len);
23     subplot(2,2,1);
24     colormap('gray');
25     imagesc(img);
26     title(['Image (Vertical, f=', num2str(f(f_len)), ')']);
27
28     subplot(2,2,2);
29     x = [-128:127];
30     y = [-128:127];
31     colormap('gray');
32     imagesc(x, y, ps);
33     title('Power Spectrum');
34
35     subplot(2,2,3);
36     x_low = [-128:127];
37     y_low = [-128:127];
38     colormap('gray');
39     imagesc(x_low, y_low, ps);
40     xlim([-lim_vals(f_len), lim_vals(f_len)]);
41     ylim([-lim_vals(f_len), lim_vals(f_len)]);

```

```

42     title('Power Spectrum (Zoom)');
43 end
44
45 %% 課題2 %%
46 f = [4, 16, 64];
47 lim_vals = [10, 30, 80];
48 for f_len = 1:length(f)
49     n = 1;
50     for l = 1:h
51         for k = 1:w
52             img(l, k) = 1 + sin(2 * pi * f(f_len) * (l-1) / h);
53         end
54     end
55     fft_image = fft2(double(img));
56     fft_shift_image = fftshift(fft_image);
57     fft_shift_abs = abs(fft_shift_image);
58     ps = power(fft_shift_abs, 2);
59
60     figure(f_len + 3);
61     subplot(2,2,1);
62     colormap('gray');
63     imagesc(img);
64     title(['Image (Horizontal, f=', num2str(f(f_len)), ')']);
65
66     subplot(2,2,2);
67     x = [-128:127];
68     y = [-128:127];
69     colormap('gray');
70     imagesc(x, y, ps);
71     title('Power Spectrum');
72
73     subplot(2,2,3);
74     x_low = [-128:127];
75     y_low = [-128:127];
76     colormap('gray');
77     imagesc(x_low, y_low, ps);

```

```

78     xlim([-lim_vals(f_len),lim_vals(f_len)]);
79     ylim([-lim_vals(f_len),lim_vals(f_len)]);
80     title('Power Spectrum (Zoom)');
81 end
82
83 %% 課題3 %%
84 H = 256;
85 W = 256;
86 set_num = [0, 60];
87 for g = 1:length(set_num)
88     img = zeros(H, W);
89     X = set_num(g);
90     Y = set_num(g);
91     img(H/2+Y -30/2:H/2+Y+30/2, W/2+X -60/2:W/2+X+60/2) = 255;
92
93     fft_img = fft2(img);
94     fft_shift_image = fftshift(fft_img);
95     fft_shift_abs = abs(fft_shift_image);
96     ps = power(fft_shift_abs, 2);
97
98     figure(g + 6);
99     subplot(1,2,1);
100    imagesc(img);
101    colormap('gray');
102    title(['Rectangle (Shift=', num2str(set_num(g)), ')']);
103
104    subplot(1,2,2);
105    imagesc(H, W, ps);
106    title('Power Spectrum');
107 end
108
109 %% 課題4 %%
110 img = imread('lena_gray.bmp');
111 [height, width] = size(img);
112 fft_img = fft2(img);
113 fft_shift_img = fftshift(fft_img);

```

```

114
115 radii = [inf, 10, 50];
116 filter_names = {'No Filter', 'R=10', 'R=50'};
117
118 for i = 1:length(radii)
119     R = radii(i);
120     fil = zeros(height, width);
121     x_center = (height / 2) + 1;
122     y_center = (width / 2) + 1;
123
124     for k = 1:height
125         for l = 1:width
126             D = sqrt(power(x_center - k, 2) + power(y_center - l, 2));
127             if D <= R
128                 fil(k,l) = 1;
129             end
130         end
131     end
132
133     fil_img = fft_shift_img .* fil;
134     ifftshift_image = ifftshift(fil_img);
135     ifft_image = ifft2(ifftshift_image);
136     filter_image = uint8(abs(ifft_image));
137
138     figure(8 + i);
139
140     subplot(2,3,1);
141     imshow(img);
142     title(['Original Image (', filter_names{i}, ')']);
143
144     subplot(2,3,2);
145     x_ax = 1:width;
146     y_ax = 1:height;
147     imagesc(x_ax, y_ax, log(1 + power(abs(fft_shift_img),2)));
148     colormap("gray");
149     title('2D Original Spectrum');

```

```

150
151     subplot(2,3,3);
152     imagesc(x_ax, y_ax, fil);
153     colormap("gray");
154     title(['Low-pass Filter (', filter_names{i}, ')']);
155
156     subplot(2,3,4);
157     imagesc(x_ax, y_ax, log(1 + power(abs(fil_img), 2)));
158     colormap("gray");
159     title('2D Filtered Spectrum');
160
161     subplot(2,3,5);
162     imshow(filter_image);
163     title(['Filtered Image (', filter_names{i}, ')']);
164 end

```